

Reinforcement Learning for Optimal Trade Execution

Building, calibrating and stress-testing an execution agent against real market microstructure

The problem

An institution that wants to buy 200,000 shares cannot simply buy them. Placing the whole order at once would exhaust available liquidity and push the price up against itself. Spreading it across the day avoids that but exposes the order to the risk that the price drifts away while it waits. Trade fast and you pay *market impact*; trade slowly and you pay *timing risk*. Choosing between them is the optimal execution problem.

Performance is measured as **implementation shortfall**: the gap between the average price actually achieved and the price at the moment the decision was made, quoted in basis points (hundredths of a percent). A shortfall of 30 bps on a \$20m order is \$60,000. At institutional volumes small percentage improvements are worth real money, which is why every large asset manager and bank runs execution algorithms.

What we are competing against

The benchmarks are not strawmen — they are the algorithms desks actually use. Understanding what each one does is necessary to read the results.

Algorithm	What it does	Weakness
TWAP	Trades an equal slice every interval, ignoring the market entirely.	Trades the same amount when liquidity is thin as when it is deep.
VWAP	Follows the historical intraday volume curve — more at the open and close, less midday.	Follows an average day, not today. Blind to current conditions.
Almgren-Chriss	Closed-form solution trading market impact against volatility risk.	Static schedule fixed in advance; cannot react to anything.
POV (x%)	Trades x% of whatever volume actually occurs in each interval.	Adapts to liquidity, but the rate itself never changes.

Why POV appears three times

Percentage-of-volume is the closest competitor, so it is tested at three speeds. A **POV 5%** algorithm takes 5% of every interval's volume — patient, low impact per trade, but slow enough that it may still be working late in the day. **POV 20%** takes a fifth of everything traded: it finishes early, paying more impact per trade but spending far less time exposed to price drift. POV 10% sits between them.

This distinction matters more than it first appears. Real intraday liquidity is U-shaped — roughly a quarter of the day's volume trades in the first half-hour and another quarter in the last. An algorithm that participates aggressively is therefore concentrating its trading in exactly the periods when trading is cheapest. **POV 20% is a genuinely hard benchmark**, and it is the one my agent does not beat.

Result

13 US large-cap equities, 488 trading days (August 2024 – July 2026), decisions once per minute, orders sized at 5% of each ticker's average daily volume. Each figure is the mean across **three independently trained policies**, evaluated on 1,000 episodes each. Percentages are reductions in execution cost relative to each benchmark.

	vs TWAP	vs VWAP	vs Almgren-Chriss	vs POV 5%	vs POV 10%	vs POV 20%
Buy orders	+28.5%	+30.1%	+30.6%	+39.6%	+23.3%	-5.8%
Sell orders	+28.4%	+26.2%	+29.0%	+38.2%	+15.4%	+0.3%
Seed std dev	±1.9-5.2	±1.6-5.1	±4.7-5.0	±3.0-4.4	±4.9-5.6	±4.8-7.7

The pattern across the POV columns is the interesting part. The agent beats a patient POV 5% by nearly 40%, a moderate POV 10% by 15–23%, and loses narrowly to an aggressive POV 20%. It has learned something close to — but not better than — an aggressive fixed participation strategy, while comfortably beating everything that trades on a schedule rather than on volume.

The limitation, stated up front: the agent does not beat POV 20%. That has held under every configuration tested — two different market impact models, with and without passive order placement, decisions at one minute and ten seconds, four impact magnitudes, every walk-forward fold and every training seed. It is the first thing a practitioner would ask about, so it leads rather than hides.

How the agent trades

The mechanism is visible in the execution statistics. The agent trades at roughly half the participation rate of the schedule-based benchmarks and pays under half the slippage per fill, while still completing every order. It is not winning by trading more cleverly at the same speed; it wins by trading less, at better moments.

It also learns **urgency-conditioned behaviour**. The agent receives an urgency input and responds to it: told to be patient it completes the order around step 311 of 390, with only 12% done by the quarter mark; told to exit immediately it finishes by step 97 with 93% done in the first quarter. A trader can ask for a different execution profile and actually get one.

How it is built

A limit order book simulator generates the market. Order flow follows a Hawkes process, so trades cluster as they do in reality; volatility switches between calm and turbulent regimes; and a multi-factor model determines how much the agent's own trading moves the price. A Gymnasium environment wraps this and a PPO agent learns the policy.

The **action** is a participation rate — what fraction of this minute's volume to take — optionally split between aggressive orders that cross the spread for a certain fill and passive orders that rest at the touch for an uncertain but cheaper one. The choice of action space turned out to be the single most consequential design decision in the project, for reasons in the next section.

Evaluation runs all seven benchmarks with transaction cost analysis, paired significance testing, walk-forward validation and per-ticker attribution. Every headline number is checked for statistical significance with a Bonferroni correction for multiple comparisons.

Grounding the simulator in real data

A synthetic simulator is only as trustworthy as its parameters, and this is where most of the work went. I bought two years of tick-by-tick data — roughly 47 million individual trades with the prevailing bid and offer at the moment of each — and used it to calibrate the model rather than assume it.

Parameter	Assumed	Measured	Evidence
Temporary impact exponent	0.60	0.65	11.75M classified trades
Permanent impact exponent	1.00	0.18	312M trade-horizon observations
Average spread (bps)	5.90	2.13	2 years, two independent methods
Order-flow clustering decay	0.30	0.013	ACF over 6,409 ticker-days
Calm-regime volatility	0.0002	0.0004	2-state mixture on rolling vol
Volatile-regime volatility	0.0012	0.0011	same

The measured impact exponent of 0.65 sits close to the square-root law that the market microstructure literature reports (~0.5), which is a reassuring independent check on the whole pipeline.

Classifying trades, and checking the classifier

Measuring market impact requires knowing which side initiated each trade — whether a buyer crossed the spread to lift the offer, or a seller hit the bid. The tape does not always say. The standard solution is the **Lee-Ready algorithm**: compare the trade price to the midpoint between bid and offer, and fall back to comparing consecutive trade prices when a trade occurs exactly at the midpoint.

My data included the exchange-reported aggressor side for a subset of trades, which let me score the classifier rather than trust it. The midpoint rule handled 89.7% of trades with **99.6% agreement**. The fallback rule used for the remainder scored **51.2%**, with a 95% confidence interval of 49.9% to 52.5% — statistically indistinguishable from guessing. Those trades were excluded from every impact measurement rather than allowed to contaminate it.

Correcting the permanent impact model

Market impact has two components. **Temporary impact** is the price concession paid to trade immediately, which decays once you stop. **Permanent impact** is the part that stays: your trading revealed information and the price does not come back. The simulator assumed permanent impact grows linearly with order size, following Kyle (1985) — the standard textbook form.

Measuring it across 312 million observations showed it is strongly **concave** instead: a fitted exponent of 0.16 to 0.19 rather than 1.0, consistent across 5, 15 and 60-minute horizons, with every size bucket statistically significant. Impact was flat across horizons, confirming it really is permanent rather than temporary impact still decaying.

I corrected the shape but deliberately did *not* import the measured magnitude. The observations sit at around 0.002% of daily volume per trade while the simulator operates at 5–25%. Extrapolating a power law across two to three orders of magnitude produced an implausible 868-fold value when I first attempted it on a related parameter, so the coefficient was instead re-anchored to match the old model at a representative order size.

The correction moved results by 3 to 5 percentage points — **less than the training noise**. Fixing the largest known error in the model barely changed the answer.

The limit of what public data can tell you

Three independent measurements converged on the same conclusion: at the sizes individual trades actually print, execution cost is driven by the spread you cross, not by how large your trade is. Only 31 of 10.5 million classified trades exceeded 1% of daily volume. Institutional orders are sliced into small pieces long before they reach the tape, so the tape does not contain observations at the scale that matters.

This means the absolute magnitude of market impact at institutional order sizes is not measurable from public data. It is a data limitation rather than an analytical one, and I handled it with sensitivity analysis — testing the result at two, five and ten times the assumed impact — rather than with false precision.

Three findings more useful than the headline

1. Six failed fixes for a problem that was not where I was looking

The agent was supposed to respond to the urgency instruction. It ignored it completely — identical behaviour whether told to be patient or to exit immediately. I spent weeks on reward shaping: scaling the incentive to trade by urgency, adding a penalty for deviating from an urgency-dependent schedule, damping that penalty during volatile regimes, damping it during thin liquidity, and finally imposing a hard structural cap on how much could be traded per step. Six attempts. Every one failed.

The cause was not the reward function at all. Logging the raw network output showed the action pinned at its maximum on every single step, and the arithmetic explained why. The action was defined as a fraction of remaining inventory, but the simulator caps trading in any one bar at 25% of that bar's volume. For a realistic order the cap bound at an action value of about 0.016 — meaning **98.4% of the action space produced identical trades**. The policy was not ignoring urgency; it had no way to express it.

Redefining the action as a participation rate made every value in the range produce a different trade, and urgency conditioning then emerged from the existing reward with no further changes. No amount of reward engineering would have found this. It required instrumenting what the policy actually output rather than reasoning about its incentives.

2. A headline that did not reproduce

Partway through, one training run scored substantially above the others and I wrote it up as the result. It did not replicate. I tested seven mechanical explanations in turn — configuration drift, a parameter that turned out to be dead code, the evaluation script, the identity of the model file, the action-space dimension, subsequent code changes, and how fees were attributed — and eliminated every one.

The actual cause was that training-seed variance was roughly four times wider than I had assumed, and that run was a favourable draw. Reinforcement learning is stochastic: the same configuration trained twice produces different policies. I had measured this early in the project and found it small, then carried the assumption forward after changing the action space in a way that made training substantially less stable.

Every figure in this document is now a mean across three independently trained policies, reported with the seed standard deviation. The single-run pattern recurred twice more, and each time the single run sat above the multi-seed mean.

3. Half the reward function was doing nothing

The reward had eight components, all added reactively while debugging other problems, none ever tested in isolation. An ablation study — removing each term, retraining from scratch, and evaluating across three seeds — gave a clear verdict.

Reward term	What it was for	Verdict
Inventory holding cost	Discourage leaving the order unfinished	Load-bearing: -6 to -12 pts
Schedule pacing	Force adherence to an urgency-dependent plan	Buys a feature (see below)
Slippage penalty	Discourage expensive fills	No measurable effect
Volatility regime penalty	Discourage trading in turbulence	No measurable effect
News-window penalty	Discourage trading around news	No measurable effect
Adverse selection penalty	Discourage being picked off	No measurable effect

Four of the six terms tested had no measurable effect and could be deleted with no loss. It is a useful corrective to the instinct that a more elaborate reward function is a better one — most of that elaboration was doing nothing except adding surface area to tune.

The measured cost of a feature

The pacing term is the interesting case. Removing it improves every single comparison and tightens the error bars — and destroys urgency conditioning completely, leaving one fixed schedule regardless of what the trader asks for.

Configuration	vs TWAP (buy)	vs Almgren-Chriss (sell)	vs POV 20% (buy)	Urgency
With pacing	+28.5% $\pm$ 5.2	+29.0% $\pm$ 4.7	-5.8% $\pm$ 7.7	works
Without pacing	+34.2% $\pm$ 4.0	+36.4% $\pm$ 4.5	+2.1% $\pm$ 2.0	dead

This is not a defect. Urgency conditioning means deliberately trading sub-optimally when told to be patient, so enforcing it must cost performance. The measured price is **5 to 8 percentage points** and wider training variance. I kept the feature, because a system that responds to a trader's instruction is more useful than one that is marginally cheaper and ignores them — but the trade-off is now quantified rather than assumed.

It is worth noting that dropping pacing is also the only configuration in which the agent beats POV 20%, and then only by a marginal +2.1%. I could have reported that number instead. Doing so would have meant quietly dropping the feature the project was partly built around, which is exactly the kind of selective reporting the rest of this document exists to guard against.

Does deciding more often help?

A natural question — and one I was asked directly — is whether the agent should decide more frequently: every second, or on every individual tick, rather than every minute. I tested it, rebuilding the entire dataset at 10-second granularity (2,340 decisions per day, 14.8 million bars) and retraining with the discount factor adjusted for the much longer horizon. Seventeen hours of training against 1.6 for the one-minute version.

No improvement. Worse on buy orders, slightly better on sells, at ten times the computational cost. There is also a structural argument against going finer still: a tick-level episode would run to roughly 60,000 steps, and the reward for executing well arrives only at the end. Assigning credit across that distance is a known hard problem in reinforcement learning, and it does not get easier with a faster simulator.

This matches how execution is organised in practice, which is in two layers: a *scheduler* deciding how much to trade over the coming minutes, and a separate *placement* layer deciding how to work each child order over milliseconds. They are distinct systems because the timescales and objectives do not mix well. This project is the first layer.

An unexpected result from that experiment

At 10-second granularity many bars contain no trades at all, and a schedule-based algorithm cannot trade through an empty bar. TWAP, VWAP, Almgren-Chriss and low-rate POV all accumulated backlog through the empty stretches and were forced to liquidate at the close. Individual episodes reached participation rates of **26 times** a bar's entire volume, at costs of 28,000 basis points.

The RL agent stayed below 0.15 throughout, because a participation-rate action naturally trades nothing when there is nothing to trade and catches up when liquidity returns. That is a real argument for volume-based over schedule-based execution, and it holds independently of any machine learning.

Engineering

Training was too slow, and both of my instincts about why were wrong. I assumed the answer was either the GPU or a more sample-efficient algorithm. Profiling found the bottleneck was the periodic evaluation callback, consuming roughly two-thirds of wall-clock time. The **GPU was measurably slower than the CPU** in every configuration tested — a small network cannot amortise the per-step data transfer — and switching algorithms would not have helped either, because environment simulation was never the constraint. Fixing the real bottleneck took a training run from eight hours to 1.6.

What this is, and what it is not

The simulator is synthetic. Its parameters are calibrated from real market data and its impact model is validated against tens of millions of real trades, but the agent has never traded against replayed order flow, and certainly never real money. The absolute magnitude of market impact is uncalibrated, and at five times the assumed impact the edge narrows to 10–17%. Results are relative comparisons within a simulator, not predictions of absolute cost.

The full limitations document runs to several pages and is published alongside the code. I think it is the more interesting half of the project: it contains the measurements that did not work, the parameters that could not be identified, and the reasoning behind every exclusion.

Code, calibration scripts and the complete audit are on GitHub. Built in Python with PyTorch and Stable-Baselines3; market data from Databento.